

VITYarathi - Build Your Own Project

StudySprint

Smart Study Tracker with Spaced-Repetition Scheduling

Course	Python Programming
Project type	Individual - Build Your Own Project
Student name	Kirti Yogi
Registration no.	24MIM10188
Faculty	Ashok Patel
Slot	B21+B22+B23+E14+E21+E22
Date	September 2026
GitHub repository	https://github.com/kirtiyogi671/studysprint

Table of Contents

- 2. Introduction
- 3. Problem Statement
- 4. Functional Requirements
- 5. Non-functional Requirements
- 6. System Architecture
- 7. Design Diagrams
- 8. Design Decisions & Rationale
- 9. Implementation Details
- 10. Screenshots / Results
- 11. Testing Approach
- 12. Challenges Faced
- 13. Learnings & Key Takeaways
- 14. Future Enhancements
- 15. References

2. Introduction

Effective learning depends on **when** material is revised, not only how long it is studied. The forgetting-curve research of Ebbinghaus showed that memory decays exponentially unless reviewed at growing intervals. **StudySprint** is a Python application that brings this idea to everyday students: it stores subjects and topics, records study sessions with a self-rated recall score, and uses the **SM-2 spaced-repetition algorithm** to decide when each topic should be revised next. A **priority queue (min-heap)** ranks due topics, and a greedy planner fits them into the study time available on a given day. Analytics such as streaks and weak topics give feedback.

The project applies core Python course concepts: functions, classes and dataclasses, modules and packages, exception handling, file and database I/O (sqlite3), the heapq, datetime, argparse, logging and csv libraries, list/dict comprehensions, and unit testing with unittest.

3. Problem Statement

Students revise inconsistently: they forget what they studied, cannot decide which topic to revisit, and have no data about where their time goes. Generic to-do lists do not model forgetting. **Objectives:**

- Provide a simple store for subjects and topics with a difficulty rating.
- Record study sessions and self-assessed recall quality.
- Automatically compute the next review date of every topic with SM-2.
- Produce a prioritised list of due topics and a plan that fits the available time.
- Report study time, streaks and weak areas; allow CSV export.

4. Functional Requirements

ID	Module	Requirement	Input -> Output
FR1	M1 Subjects & Topics	Create, read, rename, delete subjects; create, read, update difficulty, delete topics; reject duplicates.	name/title/difficulty -> stored record or error message
FR2	M2 Session Logging	Log minutes (1-720), recall quality (0-5) and notes for a topic; list recent sessions.	topic id, minutes, quality -> session row + new review date
FR3	M3 Scheduler	Apply SM-2 after each session; list due topics ordered by urgency; build a daily plan within a time budget.	available minutes -> ordered plan
FR4	M4 Analytics	Minutes per subject, last-7-day history, current streak, weakest topics, text report, CSV export.	none -> report text / CSV file
FR5	CLI workflow	Single command-line entry point with sub-commands and helpful error messages.	argv -> console output, exit code

User workflow: add subjects/topics -> ask for the daily plan -> study -> log the session with a recall score -> the system reschedules -> review analytics -> repeat (see workflow diagram).

5. Non-functional Requirements

Category	Requirement	How it is met
Performance	Any command completes in under 1 second for up to 10,000 sessions.	Indexes on topics.next_review and sessions.topic_id; SQL aggregation.
Security	No SQL injection; no secrets stored; input sanitised.	Parameterised queries only; validators.py checks every input; notes truncated to 500 chars.
Usability	Clear commands and human-readable errors, never a stack trace for bad input.	argparse help, ValidationError caught in cli.main and printed.
Reliability	Data stays consistent even if an operation fails.	Transactions via context manager, foreign keys ON, CHECK constraints.
Maintainability	Small single-purpose modules, type hints, docstrings, tests.	10 modules under studysprint/, 26 unit tests.
Logging	Important events and all SQL failures are recorded.	Rotating file log (logs/studysprint.log).
Portability	Runs on Windows/Linux/macOS with no third-party runtime dependency.	Standard library only.

6. System Architecture

StudySprint uses a layered architecture: a thin presentation layer (CLI), a business-logic layer made of four functional modules plus validators, and a data layer that wraps SQLite. Logging and configuration are cross-cutting concerns. Dependencies point downwards only, so any layer can be replaced (for example a GUI instead of the CLI) without touching the others.

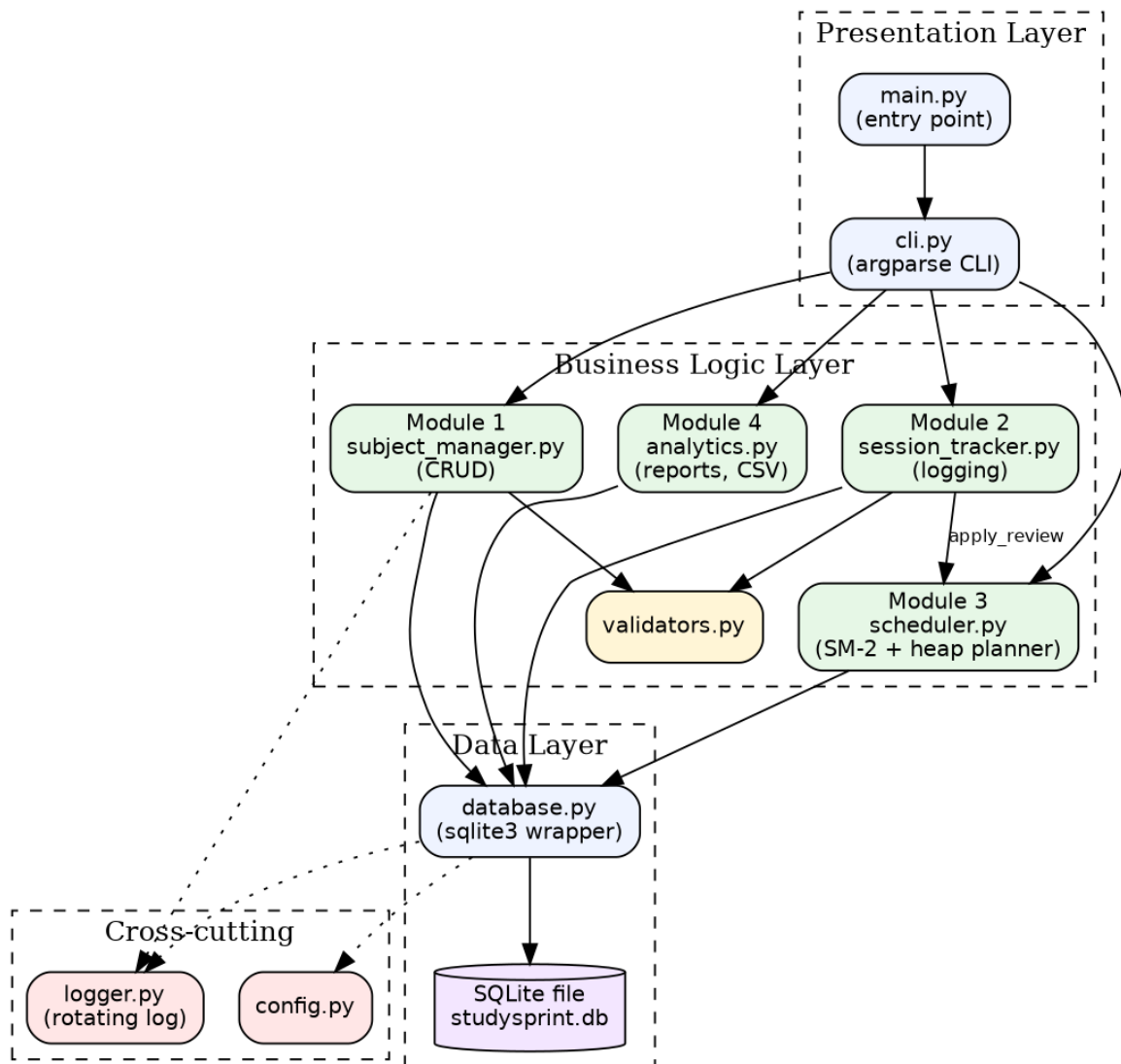


Figure 1 - System architecture

7. Design Diagrams

7.1 Use Case Diagram

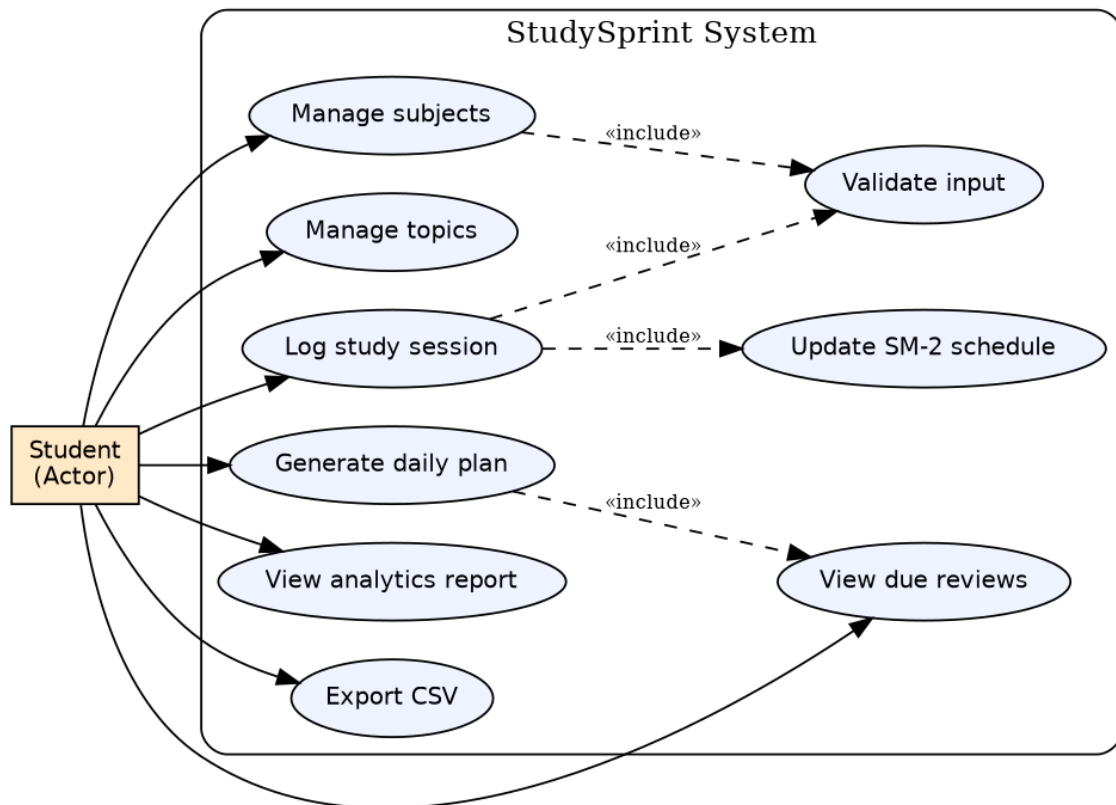


Figure 2 - Use case diagram

7.2 Workflow Diagram

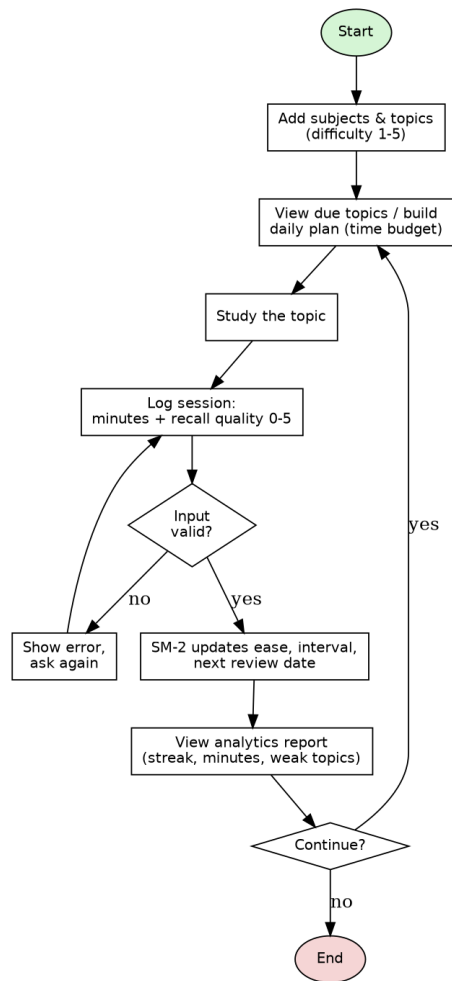


Figure 3 - Workflow of a study cycle

7.3 Sequence Diagram - logging a session

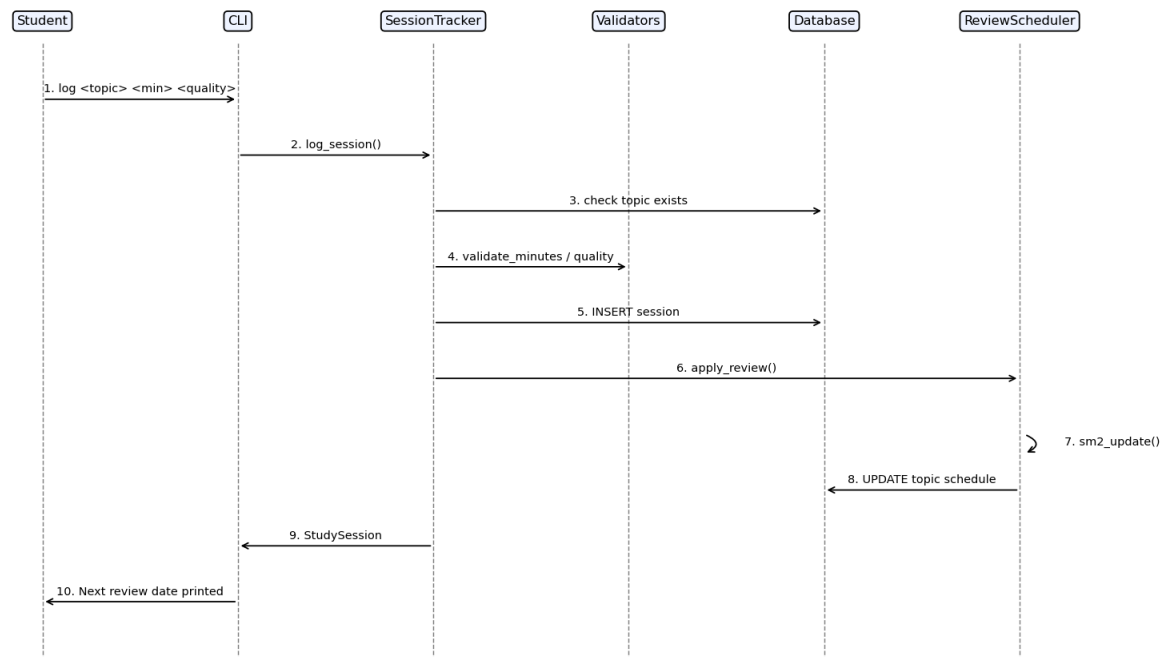


Figure 4 - Sequence diagram for the 'log' command

7.4 Class Diagram

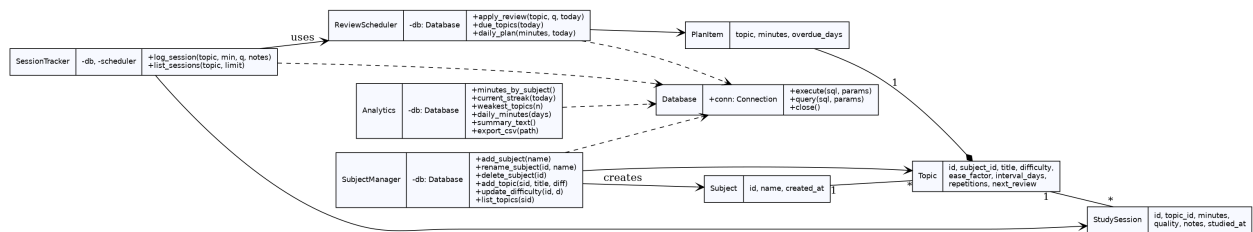


Figure 5 - Class diagram

7.5 ER Diagram and Schema

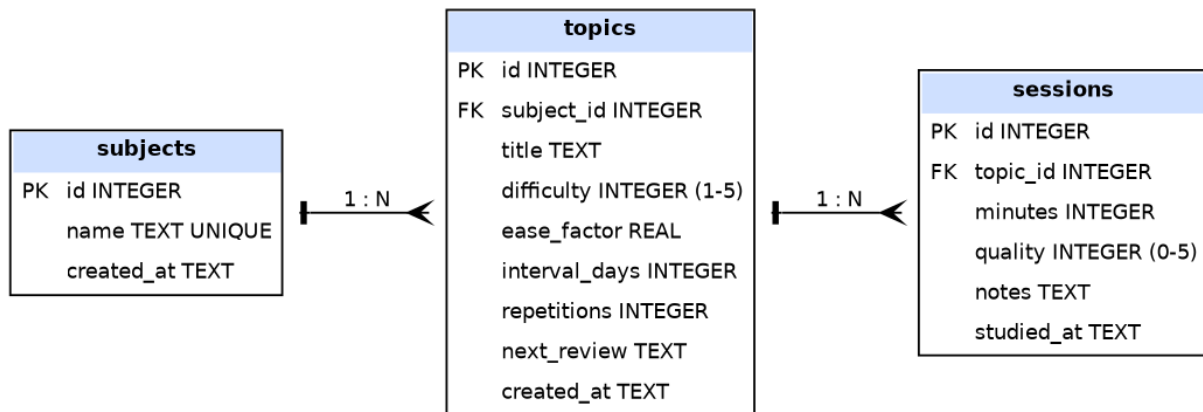


Figure 6 - ER diagram (SQLite)

Schema: **subjects**(id PK, name UNIQUE NOCASE, created_at); **topics**(id PK, subject_id FK ON DELETE CASCADE, title, difficulty CHECK 1-5, ease_factor, interval_days, repetitions, next_review, UNIQUE(subject_id,title)); **sessions**(id PK, topic_id FK ON DELETE CASCADE, minutes CHECK >0, quality CHECK 0-5, notes, studied_at). Deleting a subject removes its topics and sessions automatically.

8. Design Decisions & Rationale

Decision	Rationale
SQLite instead of JSON/CSV files	Relational integrity (FK, UNIQUE, CHECK), SQL aggregation for analytics, atomic transactions; still zero-install.
SM-2 algorithm	Well-known, simple, needs only ease/interval/repetitions per item; easy to unit-test as a pure function.
Min-heap for due topics	Priority = (most overdue, hardest, id). A heap gives $O(n \log n)$ ordering and shows practical use of a data structure.
Greedy time-budget planner	Topics are taken in priority order if they fit the remaining minutes; simple and predictable (a knapsack solver would be over-engineering).
Dependency injection (Database passed into each manager)	Tests use an in-memory database; modules stay decoupled.
Dates passed as parameters (today=...)	Makes time-dependent logic (streaks, due dates) deterministic in tests.
CLI first	Fastest way to a complete, testable core; the layered design allows a GUI later.

SM-2 rules used: if quality < 3 the repetition count resets and the interval becomes 1 day; otherwise interval = 1 day (1st success), 6 days (2nd) and $\text{round}(\text{previous interval} \times \text{ease})$ afterwards. Ease is updated by $\text{EF}' = \text{EF} + 0.1 - (5-q)(0.08 + (5-q)0.02)$ and never falls below 1.3.

9. Implementation Details

File	Responsibility
main.py	Entry point calling cli.main().
studysprint/config.py	Paths and constants; environment overrides.

studysprint/logger.py	Rotating file logger with safe fallback.
studysprint/models.py	Dataclasses Subject, Topic, StudySession.
studysprint/validators.py	ValidationError and validate_* functions.
studysprint/database.py	SQLite connection, schema creation, execute/query helpers with logging.
studysprint/subject_manager.py	Module 1 - CRUD for subjects and topics.
studysprint/session_tracker.py	Module 2 - session logging; triggers scheduler.
studysprint/scheduler.py	Module 3 - sm2_update(), ReviewScheduler (due_topics with heapq, daily_plan).
studysprint/analytics.py	Module 4 - aggregates, streak, report, CSV export.
studysprint/cli.py	argparse sub-commands and error handling.
tests/	7 test files covering validators, all modules and an end-to-end CLI flow.

Core code - SM-2 update (scheduler.py):

```
def sm2_update(ease, interval, repetitions, quality):
    if quality < 3:                                # failed recall
        repetitions, interval = 0, 1
    else:
        if repetitions == 0: interval = 1
        elif repetitions == 1: interval = 6
        else: interval = max(1, round(interval * ease))
        repetitions += 1
    ease = ease + 0.1 - (5-quality)*(0.08 + (5-quality)*0.02)
    return max(MIN_EASE, round(ease, 3)), interval, repetitions
```

Priority queue: each due topic is pushed as (-overdue_days, -difficulty, id, topic) onto a heap and popped in order, so the most overdue and hardest topics come first. **Error handling:** validators raise ValidationError, database IntegrityErrors are translated to friendly messages, and cli.main prints them with exit code 1.

10. Screenshots / Results

```
$ python main.py add-subject Python
Added subject #1: Python
$ python main.py add-subject Data Structures
Added subject #2: Data Structures
$ python main.py add-topic 1 Decorators -d 4
Added topic #1: Decorators (difficulty 4)
$ python main.py add-topic 1 List Comprehensions -d 2
Added topic #2: List Comprehensions (difficulty 2)
$ python main.py add-topic 2 Binary Heaps -d 5
Added topic #3: Binary Heaps (difficulty 5)
```

Figure 7 - Adding subjects and topics

```
$ python main.py list-topics
#1 Decorators diff=4 next=new
#2 List Comprehensions diff=2 next=new
#3 Binary Heaps diff=5 next=new
$ python main.py due
#3 Binary Heaps (diff 5)
#1 Decorators (diff 4)
#2 List Comprehensions (diff 2)
$ python main.py plan 60
35 min #3 Binary Heaps (overdue 0d)
20 min #2 List Comprehensions (overdue 0d)
Planned 55/60 min.
```

Figure 8 - Due list and 60-minute daily plan (hardest topic first)

```
$ python main.py log 1 40 4 -n Good grasp
Logged 40 min. Next review: 2026-09-30 (interval 1d, ease 2.5)
$ python main.py log 3 50 2
Logged 50 min. Next review: 2026-09-30 (interval 1d, ease 2.18)
$ python main.py log 2 20 5
Logged 20 min. Next review: 2026-09-30 (interval 1d, ease 2.6)
$ python main.py due
Nothing due - great job!
```

Figure 9 - Logging sessions; SM-2 reschedules (poor recall lowers ease to 2.18)

```
$ python main.py report
=== StudySprint Report ===
Total study time : 110 min
Current streak   : 1 day(s)

Minutes per subject:
  Python          60 min #####
  Data Structures 50 min #####

Last 7 days:
  2026-09-23      0 min
  2026-09-24      0 min
  2026-09-25      0 min
  2026-09-26      0 min
  2026-09-27      0 min
  2026-09-28      0 min
  2026-09-29     110 min

Topics needing attention:
  Data Structures / Binary Heaps (avg recall 2.0/5)
  Python / Decorators (avg recall 4.0/5)
  Python / List Comprehensions (avg recall 5.0/5)
$ python main.py log 1 30 9
Error: Recall quality must be between 0 and 5.
```

Figure 10 - Analytics report and validation error handling

11. Testing Approach

Tests use Python's `unittest` (also runnable with `pytest`) and an in-memory SQLite database, so they are fast and isolated. **Result: 26 tests, all passing.**

Test file	What is verified
test_validators.py	Boundaries and invalid types for name, difficulty, quality, minutes, date.
test_subject_manager.py	CRUD, case-insensitive duplicate rejection, cascade delete, bad ids.
test_scheduler.py	SM-2 interval progression (1, 6, 16), reset on failure, ease floor, priority order, plan never exceeds budget.
test_session_tracker.py	Session insert updates next_review and repetitions; invalid inputs rejected; ordering.
test_analytics.py	Aggregations, streak (today/yesterday/broken), weakest topic, CSV export, empty database.
test_cli.py	End-to-end flow add -> plan -> log -> due -> report, and error propagation.

```
$ python -m unittest discover -s tests -v
test_empty_db [analytics.TAnalytics.test_empty_db) ... ok
test_export_and_summary [analytics.TAnalytics.test_export_and_summary) ... /home/claude/study
  self.assertEqual(len(open(p).read().strip().splitlines()), 4)
ResourceWarning: Enable tracemalloc to get the object allocation traceback
ok
test_minutes_by_subject [analytics.TAnalytics.test_minutes_by_subject) ... ok
test_streak [analytics.TAnalytics.test_streak) ... ok
test_weakest_and_daily [analytics.TAnalytics.test_weakest_and_daily) ... ok
test_end_to_end_flow [cli.TCLI.test_end_to_end_flow) ... ok
test_errors_propagate [cli.TCLI.test_errors_propagate) ... ok
test_ease_floor [scheduler.TSM2.test_ease_floor) ... ok
test_estimate [scheduler.TSM2.test_estimate) ... ok
test_failure_resets [scheduler.TSM2.test_failure_resets) ... ok
test_interval_progression [scheduler.TSM2.test_interval_progression) ... ok
test_apply_review_sets_next_date [scheduler.TScheduler.test_apply_review_sets_next_date) ...
test_daily_plan_respects_budget [scheduler.TScheduler.test_daily_plan_respects_budget) ... ok
test_new_topics_are_due [scheduler.TScheduler.test_new_topics_are_due) ... ok
test_priority_order_most_overdue_first [scheduler.TScheduler.test_priority_order_most_overdue
test_list_sessions_order [session_tracker.TSessionTracker.test_list_sessions_order) ... ok
test_log_updates_schedule [session_tracker.TSessionTracker.test_log_updates_schedule) ... ok
test_validation [session_tracker.TSessionTracker.test_validation) ... ok
test_bad_inputs [subject_manager.TSubjectManager.test_bad_inputs) ... ok
test_crud_subject [subject_manager.TSubjectManager.test_crud_subject) ... ok
test_duplicate_subject_case_insensitive [subject_manager.TSubjectManager.test_duplicate_subje
test_topics_and_cascade [subject_manager.TSubjectManager.test_topics_and_cascade) ... ok
test_date [validators.TValidators.test_date) ... ok
test_name_empty_or_long [validators.TValidators.test_name_empty_or_long) ... ok
test_name_ok_and_trimmed [validators.TValidators.test_name_ok_and_trimmed) ... ok
test_ranges [validators.TValidators.test_ranges) ... ok

-----
Ran 26 tests in 0.049s

OK
```

Figure 11 - Unit test run

12. Challenges Faced

- **Testing time-dependent logic:** streaks and due dates depend on today's date; solved by passing *today* as an argument.
- **In-memory SQLite:** each new connection creates a fresh database, so one persistent connection is held in the Database class.
- **Choosing a fair priority:** combining overdue days and difficulty into a heap key needed experimentation.

- **Friendly errors:** translating low-level `sqlite3.IntegrityError` messages into understandable feedback.
- **Foreign keys:** SQLite disables them by default; `PRAGMA foreign_keys=ON` is required for cascade deletes.

13. Learnings & Key Takeaways

- Designing layered, loosely coupled modules makes testing much easier.
- Small pure functions (`sm2_update`) are the easiest and most reliable code to test.
- Constraints in the database (CHECK, UNIQUE, FK) act as a second line of defence behind validation.
- Using Git commits per module gives a clear development history.
- Real algorithms and data structures (SM-2, heap) can be applied to everyday problems.

14. Future Enhancements

- Tkinter or web (Flask) front-end with charts.
- Flashcard content per topic and quiz mode.
- Desktop/e-mail reminders for due reviews.
- Multi-user accounts and cloud sync.
- Machine-learning model to predict forgetting from session history.
- Export analytics to PDF.

15. References

- Wozniak, P. A. (1990). *Optimization of learning* - the SM-2 algorithm, SuperMemo.
- Ebbinghaus, H. (1885). *Memory: A Contribution to Experimental Psychology*.
- Python Software Foundation. Python 3 documentation: `sqlite3`, `heapq`, `argparse`, `logging`, `unittest`. <https://docs.python.org/3/>
- SQLite documentation. <https://www.sqlite.org/docs.html>
- Fowler, M. *UML Distilled*. Addison-Wesley.